

Practica #2 Clase que utilice un objeto que contenga argumentos

En este ejemplo la clase addItem utiliza un apuntador a la referencia donde se encuentra el nuevo item y regresa un float con el nuevo total.

El item se pasa como apuntador, ya que al terminar la ejecución de la función addItem los objetos argumentos se destruyen y seria necesario duplicar el item en la memoria estática (HEAP).

Código Fuente

```
1  /*****
2  Instituto Tecnológico de Durango
3  Departamento Metal-Mecanica
4
5  Programacion Avanzada 7U
6  Fabian Quiones Escarzaga 23040871
7  Ing. Mario Gerardo Hernandez Marines
8  2026-03-03
9  Practica 2 U1 Clase que utilice un Objeto que contenga argumentos
10 *****/
11
12 #include <iostream>
13 #include <cstring>
14 #include <math.h>
15 #include <vector>
16
17 using namespace std;
18 //Precio en centavos
19 class item {
20     friend class order;
21     protected:
22         int m_precio = 0;
23         char * m_name;
24     public:
25         item(int precio, const char* name) : m_precio(precio) {
26             m_name = new char[strlen(name)+1];
27             strcpy(m_name, name);
28         };
29         const char * getName(){ return(m_name);};
30         float getPrice(){ return(m_precio/100.00f);};
31 };
32
33 class client{
34     protected:
35         char * m_name;
36         char * m_addr;
37     public:
38         client(const char * name, const char * addr){
39             m_name = new char[strlen(name)+1];
40             strcpy(m_name, name);
41             m_addr = new char[strlen(addr)+1];
42             strcpy(m_addr, addr);
43         };
44 };
45
46 class order : public client{
47     private:
48         std::vector<item*> m_items;
49         int m_nettotal=0;
```

```

50 public:
51     order(const char * name, const char * addr) : client(name,addr){ };
52
53     float addItem(item* newItem){ //para no duplicar el objeto newItem lo mejor es
54         utilizar su valor de apuntador float addItem(item * newItem) y newItem->
55         m_precio
56         m_items.push_back(newItem);
57         m_netttotal += newItem->m_precio;
58         return(m_netttotal/100.00f);
59     };
60
61     float removeItem(unsigned int idx){
62         m_netttotal -= m_items[idx]->m_precio;
63         m_items.erase(m_items.begin()+idx);
64         return(m_netttotal/100.00f);
65     };
66
67     void printLabel(const char * trackingNum){
68         cout << trackingNum << endl;
69         cout << "Destinatario: \n \t" << m_name << endl << m_addr << endl;
70         cout << "Manifiesto de Articulos:\n";
71         for(item * articulo : m_items){
72             cout << "\t" << articulo->m_name << " $"<< articulo-> m_precio/100.00f
73             << endl;
74         };
75         cout << "-----\n" << "Total: $"<<
76             m_netttotal/100.00f;
77     };
78 };
79
80 void print_items(vector<item*>* itemList){
81     cout << "#ID | Nombre del Articulo | Precio"<<endl;
82     for(int i=0; i<itemList->size(); i++){
83         item* art = itemList->at(i);
84         cout<< "\t "<< i << " | " <<art->getName() << " |$"<< art->getPrice()<< endl;
85     };
86 }
87
88 int main(){
89     vector<item*> availableItems = {
90         new item(10000, "Calculadora CASIO fx-ms10"),
91         new item(2000, "Libreta Cuadrcula Grande"),
92         new item(100, "Borrador Pelikan"),
93         new item(10000, "Pluma Pilot Blanca"),
94         new item(5000, "Sacapuntas Hexagonal")
95     };
96     cout << "PORTAL DE ORDENES DE USUARIO: \n IDENTIFIQUESE:\n\tNombre Completo: ";
97     char name[256];
98     cin >> name;
99     cout << "\n";
100     cout << "";
101     cout << "";
102     cout << "";
103     cout << "\n\tDireccion: ";
104     char addr[256];
105     cin >> addr;
106     cout << "\x1b[2J\x1b[1;1H";//Limpia la Consola
107     cout << "Articulos disponibles:\n";
108     print_items(&availableItems);

```

```

105     cout << "PORTAL DE USUARIO\n ORDEN #1 \n Cantidad de articulos desea agregar: "
        ;
106     order newOrder(name,addr);
107     unsigned int qty=0;
108     cin >> qty;
109     for(int i = 0; i< qty; i++){
110         print_items(&availableItems);
111         cout << "Seleccione Producto: ";
112
113         int choosedIdx =-1;
114         cin >> choosedIdx;
115         if(choosedIdx<0 || choosedIdx >= availableItems.size()){cout << "OPCION
            INVALIDA!!\n"; cin >> choosedIdx;}
116         else{
117             cout << "\x1b[2J\x1b[1;1H";//Limpia la Consola
118             cout << "Total acumulado (SIN IVA): "<< newOrder.addItem(availableItems
                [choosedIdx])<<endl<<endl<<"
                -----"<<endl;
119         };
120     };
121
122     cout << endl;
123     newOrder.printLabel("0xDEADBEEF");
124 }

```

Registro de Ejecución (Interacción)

```

PORTAL DE ORDENES DE USUARIO:
  IDENTIFIQUESE:
    Nombre Completo: Fabian

    Direccion: Durango

    Articulos disponibles:
#ID / Nombre del Articulo / Precio
  0| Calculadora CASIO fx-ms10 |$100
  1| Libreta Cuadrícula Grande |$20
  2| Borrador Pelikan |$1
  3| Pluma Pilot Blanca |$100
  4| Sacapuntas Hexagonal |$50
PORTAL DE USUARIO
  ORDEN #1
    Cantidad de articulos desea agregar: 3
#ID / Nombre del Articulo / Precio
  0| Calculadora CASIO fx-ms10 |$100
  1| Libreta Cuadrícula Grande |$20
  2| Borrador Pelikan |$1
  3| Pluma Pilot Blanca |$100
  4| Sacapuntas Hexagonal |$50
Seleccione Producto: 0

Total acumulado (SIN IVA): 100

```

#ID / Nombre del Articulo / Precio
0| Calculadora CASIO fx-ms10 |\$100
1| Libreta Cuadrícula Grande |\$20
2| Borrador Pelikan |\$1
3| Pluma Pilot Blanca |\$100
4| Sacapuntas Hexagonal |\$50
Seleccione Producto: 1

Total acumulado (SIN IVA): 120

#ID / Nombre del Articulo / Precio
0| Calculadora CASIO fx-ms10 |\$100
1| Libreta Cuadrícula Grande |\$20
2| Borrador Pelikan |\$1
3| Pluma Pilot Blanca |\$100
4| Sacapuntas Hexagonal |\$50
Seleccione Producto: 2

Total acumulado (SIN IVA): 121

0xDEADBEEF
Destinatario:
 Fabian
Durango
Manifiesto de Articulos:
 Calculadora CASIO fx-ms10 \$100
 Borrador Pelikan \$1
 Libreta Cuadricula Grande \$20

Total: \$121.00
